UD1 - UD2

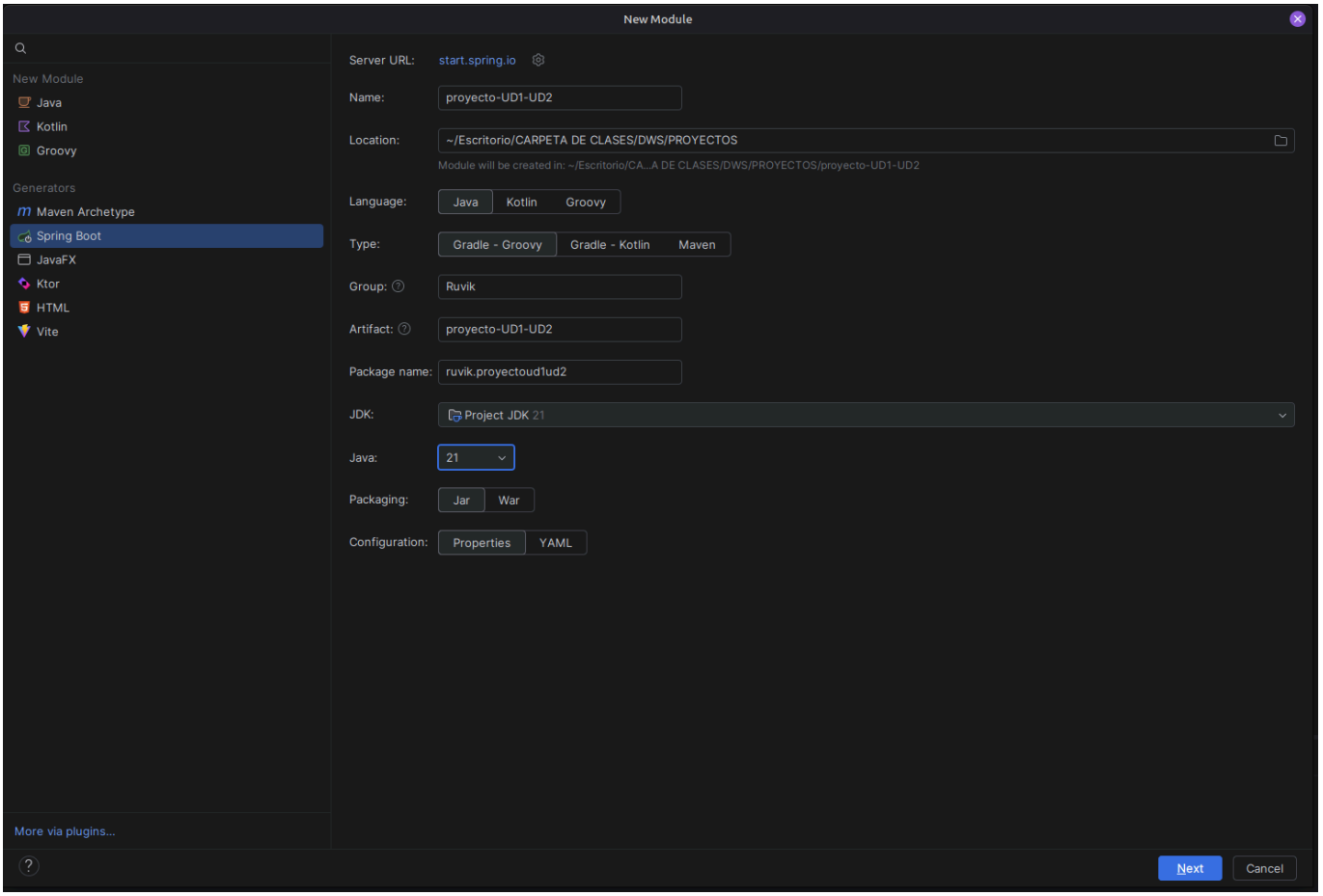
KERIN AGUILERA

INDICE

- UD1 - UD2
  - KERIN AGUILERA
    - INDICE
    - PRIMERO INTENTO EL FALLIDO
    - SEGUNDO INTENTO, EL BUENO
      - PAGINA ESTATICA
        - Sring Initializr
  - Conclusión
- FINAL

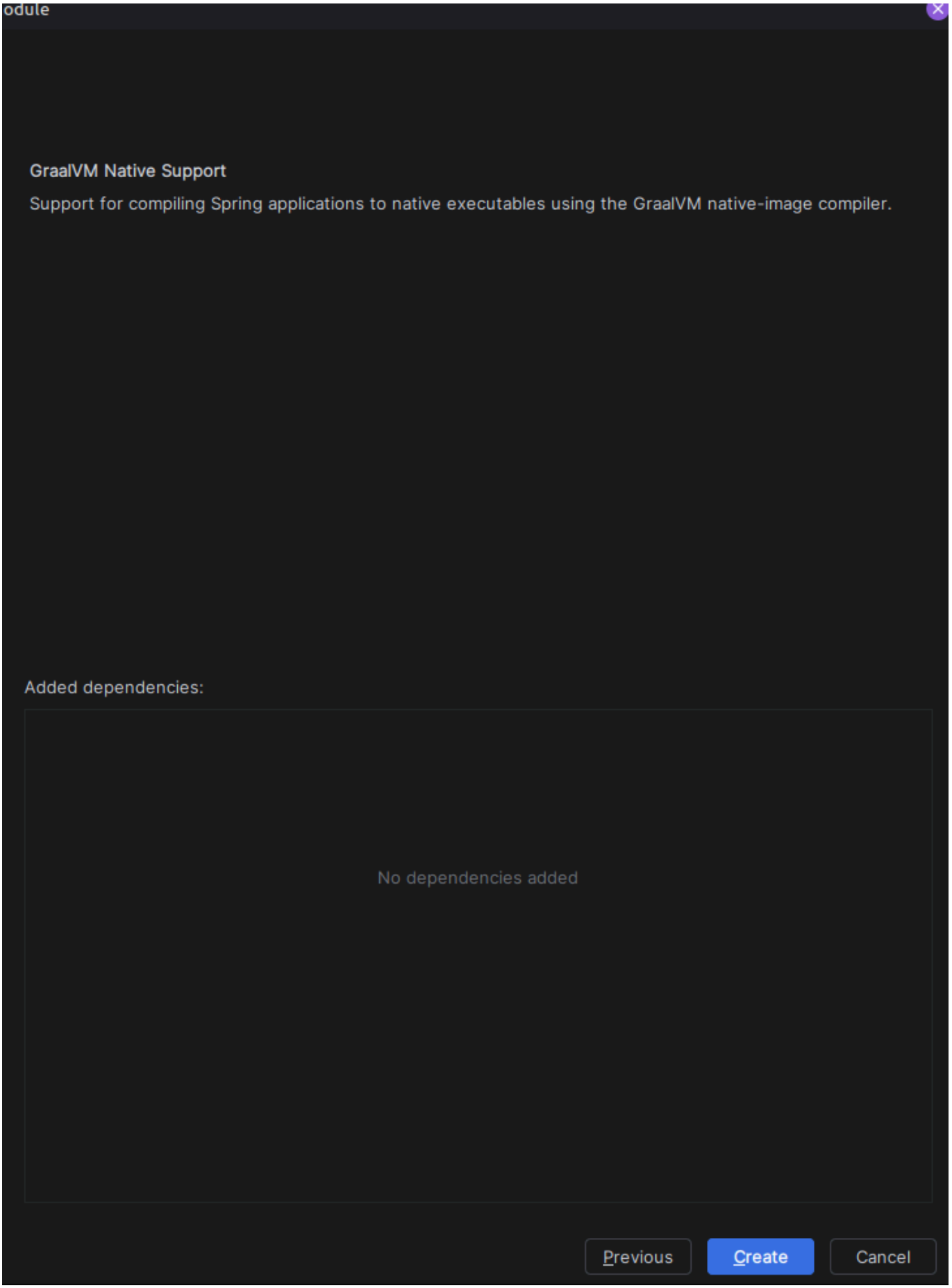
PRIMERO INTENTO EL FALLIDO

Siguiente el hilo de la practica, pide que intentemos crear un proyecto **springboot** de manera vanila, sin modificaciones e sin incluir nada.



Aqui solo haremos un cambio que es elegir Java 21.

Luego, le damos next y pasamos a la parte de la dependecias, igual no incluimos nada en ellas.



Ahora la cuestión es ¿Funciona esto? Pues de momentos nos da todo lo que puede tener una apps springboot.

tenemos la clase inical

```java
package ruvik.proyectoud1ud2;

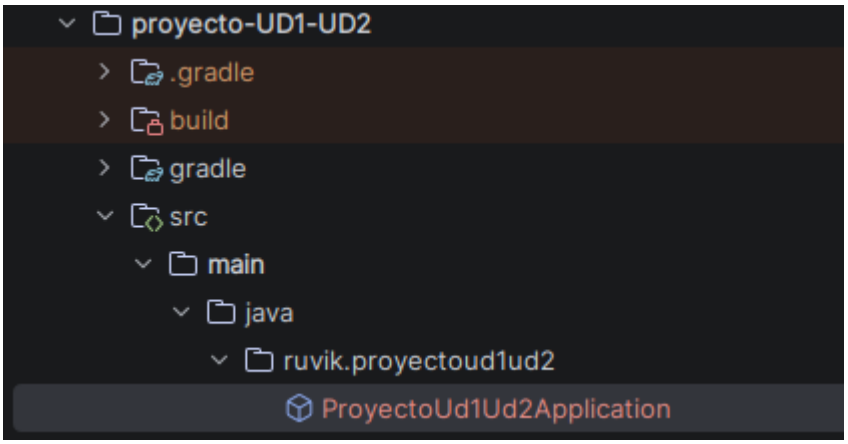
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class ProyectoUd1Ud2Application {

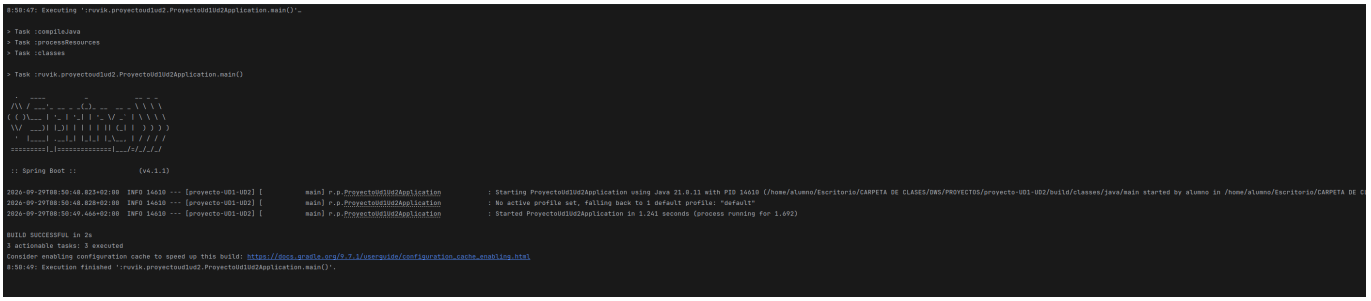
    public static void main(String[] args) {
        SpringApplication.run(ProyectoUd1Ud2Application.class, args);
    }

}
```

Con su estructura.



Es curioso al intentar ejecutar lo que es el programa sin tener nada adentro, no se queda ejecutando, solo devuelve algo por terminal y finaliza.



Como era de esperar y es logico cuando intenamos incluir las anocaciones de Spring | Quickstart fallan, salta por los aires y es que **no hemos puestos las dependecias necesarias** que sorportan el tema de la web.

ProyectoUd1Ud2Application.java

```java
package ruvik.proyectoud1ud2;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
@RestController
public class ProyectoUd1Ud2Application {

    public static void main(String[] args) {
        SpringApplication.run(ProyectoUd1Ud2Application.class, args);
    }

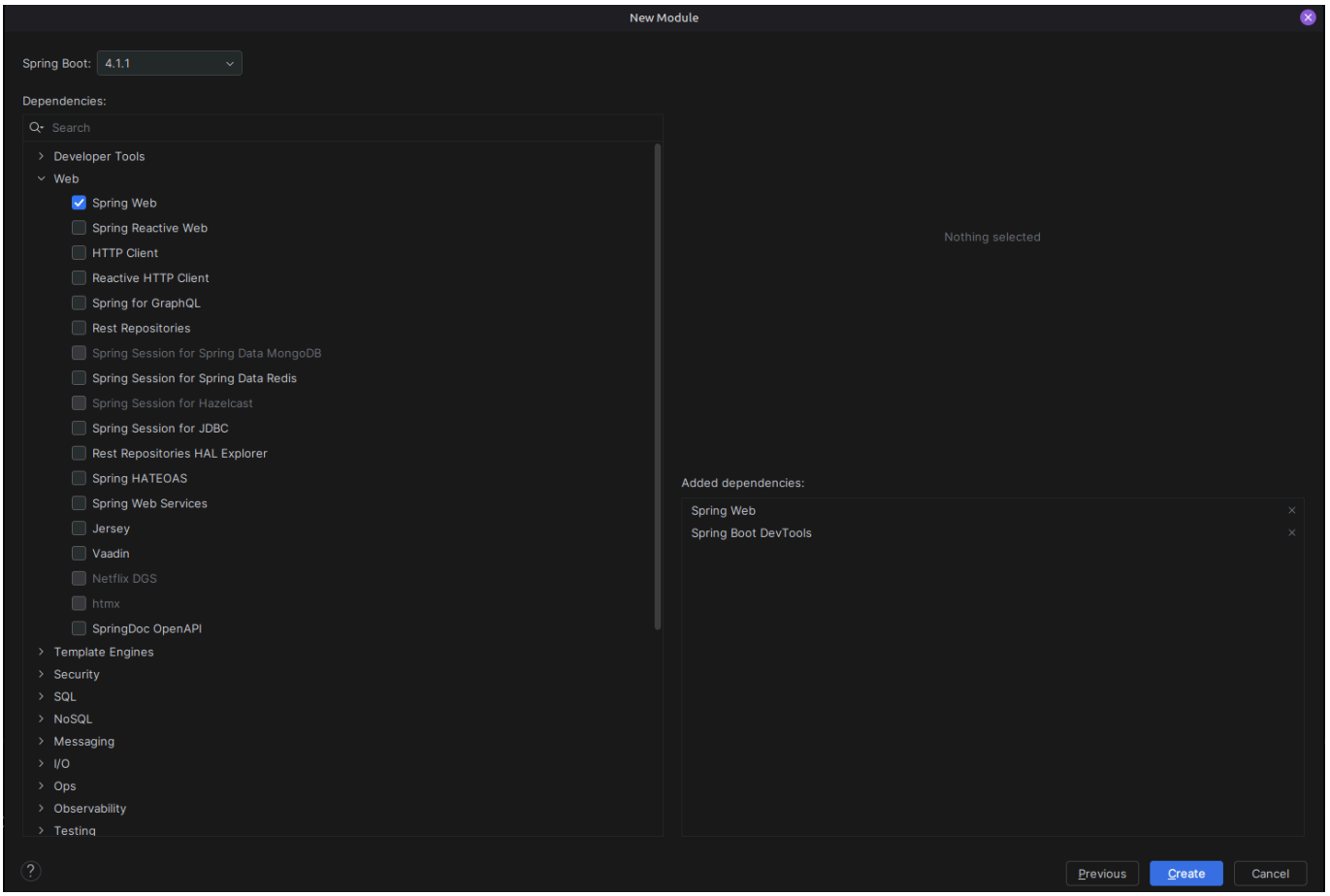
    @GetMapping("/hello")  no usages
    public String hello(@RequestParam(value = "name", defaultValue = "World") String name) {
        return String.format("Hello %s!", name);
    }

}
```

CONCLUSIÓN TODO ESTO FALLA

SEGUNDO INTENTO, EL BUENO

Ahora volvemos a inicializar un proyecto como antes, pero ahora la peculiaridad es añadir una dependencia que lo cambia todo. `Spring web`



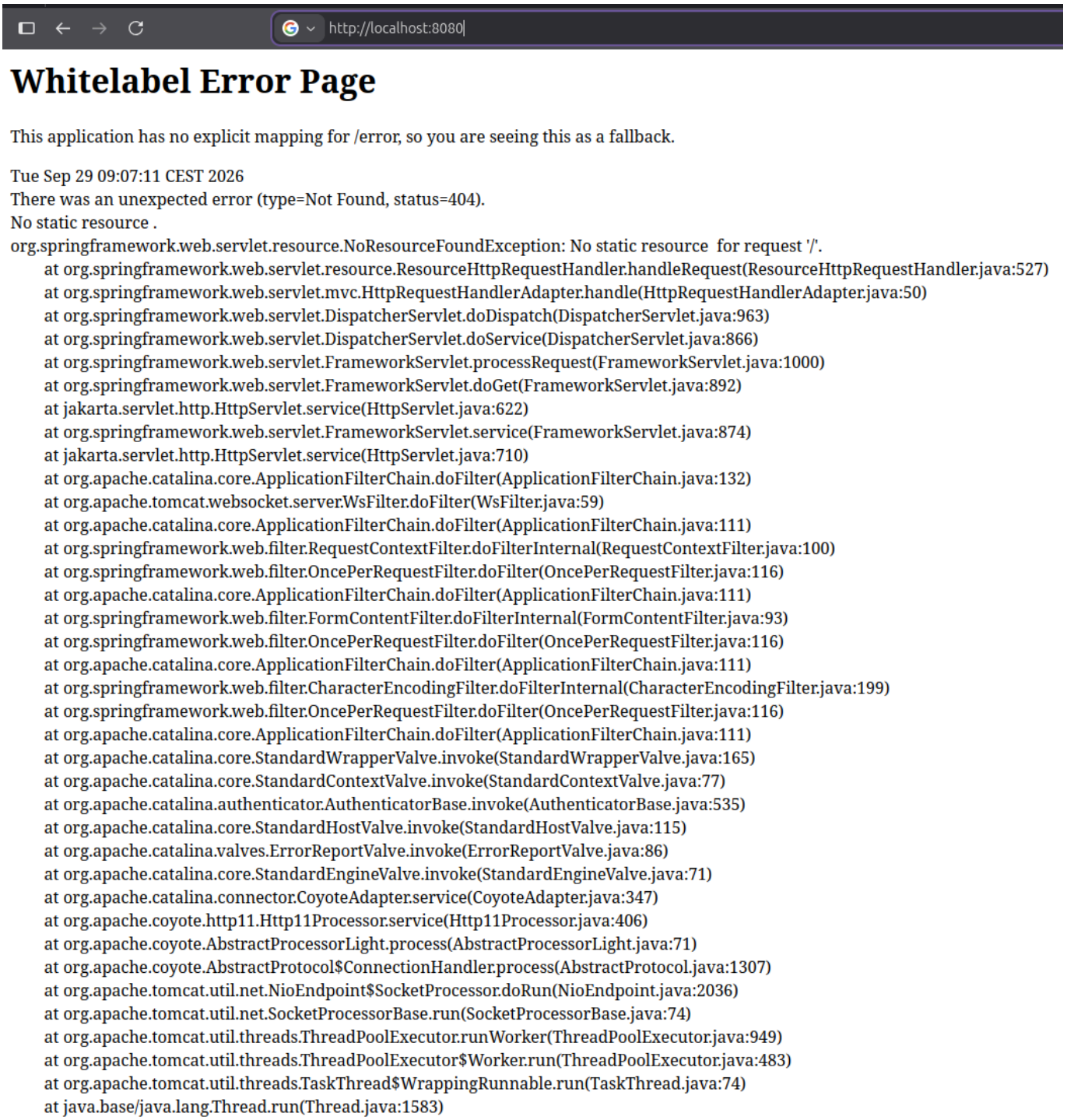
ahora con esto ya podemos comprar lo que sucedido en la anterior y sacar conclusiones. Y es que ahora si permite poner las anotaciones y los imports, quitando los imports sugiere lo mismo, porque aun no tenemos dependecias como JPA o mas que si tiene 2 imports distinos y al ejeuctar el proyecto funciona, no se detiene sino que se mantiene en ejecución, asi como ahora si nos dice que podemos ver lo que muestra en localhost.

```
  .   ____          _            __ _ _
 /\\ / ___'_ __ _ _(_)_ __  __ _ \ \ \ \
( ( )\___ | '_ | '_| | '_ \/ _` | \ \ \ \
 \\/  ___)| |_)| | | | | || (_| |  ) ) ) )
  '  |____| .__|_| |_|_| |_\__, | / / / /
 =========|_|==============|___/=/_/_/_/

 :: Spring Boot ::                (v4.1.1)

2026-09-29T09:05:24.499+02:00  INFO 21074 --- [UD1-UD2] [  restartedMain]
o.s.boot.tomcat.TomcatWebServer          : Tomcat initialized with port
8080 (http)
2026-09-29T09:05:24.512+02:00  INFO 21074 --- [UD1-UD2] [  restartedMain]
o.apache.catalina.core.StandardService   : Starting service [Tomcat]
```

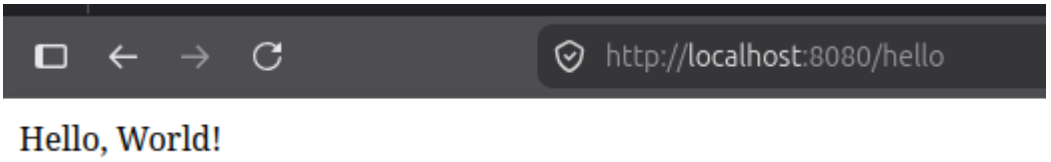
Y ya no solo eso, sino que ahora si vamos a nuestros navegador y y ponemos `localhost:8080` ya nos responde algo



¿Pero que ocurre? Pues el fallos es claramente logico y normal, en nuestro proyecto tenemos la siguiente lineas

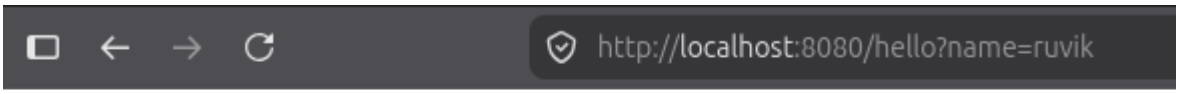
```
    @GetMapping("/hello")
    public String hello(@RequestParam(value = "name", defaultValue =
"World") String name) {
        return String.format("Hello, %s!", name);
    }
```

que nos dice que para visualizar *Hello World* debemos ir a la ruta `/hello` asi que en el buscador deberias de poner `localhost:8080/hello` y ahora si esto debria devolvernos lo siguiente:



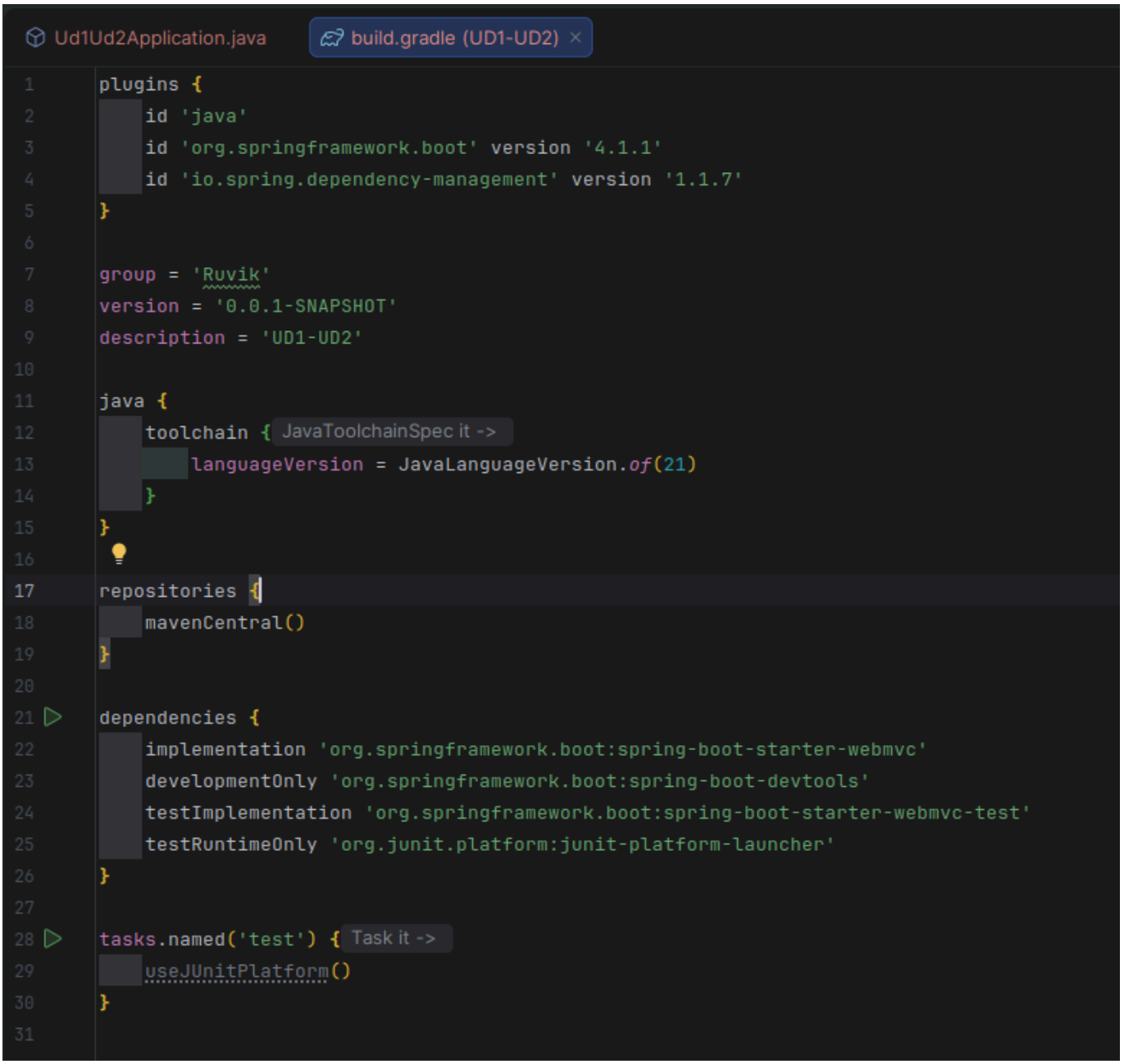
Hello, World!

Y es que ahora con la pequeña logica que hay dentro, si nosotros en la url introducimos un valor en la variable name, debe saludarnos `http://localhost:8080/hello?name=ruvik`



Hello, ruvik!

Ya tengo la depedencias DevTools pero viendo dentro de gradle tenemos lo siguiente



en ella tenemos lo siguiente

```
plugins {
    id 'java'
    id 'org.springframework.boot' version '4.1.1'
    id 'io.spring.dependency-management' version '1.1.7'
}
```

Esto en donde determinamos versión del framework asi como el propio gradle que gestionas sus paquetes.

```
group = 'Ruvik'
version = '0.0.1-SNAPSHOT'
description = 'UD1-UD2'

java {
```

```
    toolchain {
        languageVersion = JavaLanguageVersion.of(21)
    }
}

repositories {
    mavenCentral()
}
```

aqui esta todo lo que dejamos por default, viendo tambien que esta el la versión 21 de Java que elegimos

```
dependencies {
    implementation 'org.springframework.boot:spring-boot-starter-webmvc'
    developmentOnly 'org.springframework.boot:spring-boot-devtools'
    testImplementation 'org.springframework.boot:spring-boot-starter-
webmvc-test'
    testRuntimeOnly 'org.junit.platform:junit-platform-launcher'
}
```

aqui esta donde nos moveremos mas, las dependecias del proyecto, donde a manos podemos añadir nuevas dependecias, o buscar dependencias saviendo si funcionan o no.

> ANOTANCIÓN: todo lo que he dicho antes es segundo entiendo lo que veo. Puede existir fallos.

Al modificar tanto nombre pasar al español a ingles y sigue funcionando

```
package ruvik.ud1ud2;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;


@SpringBootApplication
@RestController
public class Ud1Ud2Application {

    public static void main(String[] args) {
        SpringApplication.run(Ud1Ud2Application.class, args);
    }

    @GetMapping("/hola")
    public String hello(@RequestParam(value = "nombre", defaultValue =
"Mundo") String name) {
        return String.format("Hola, %s!", name);
```

```
    }

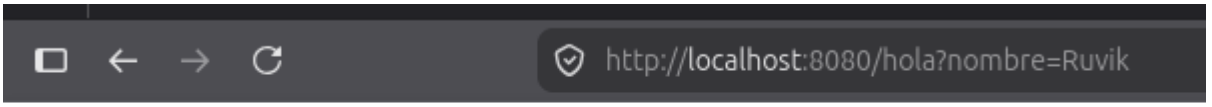
}
```

Vemos el cambio de ingles a español

Y aqui abajos las repuesta de la petención `localhost:8080/hola` y `localhost:8080/hola?nombre=ruvik`



Hello, World!



Hola, Ruvik!

Si funciona si pones atonaciones HTML dentro de la repuesta

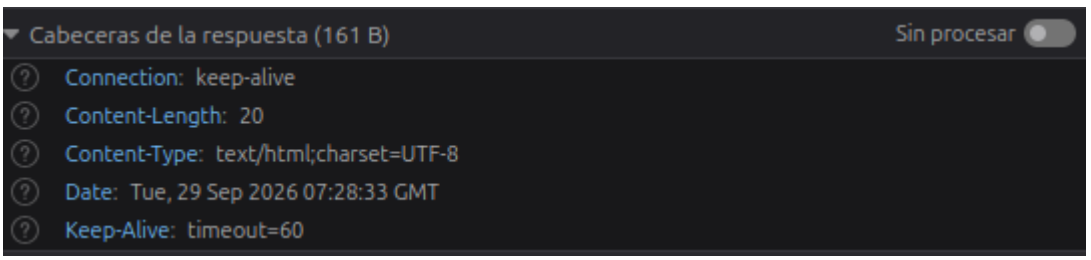
```
return String.format("Hola, <b> %s!</b>",name);
```

Si refrescamos la pagina web asi es como lo devuelve



Hola, **Ruvik!**

Y si vemos la repuesta de la cabecera esto es lo que tenemos



PAGINA ESTATICA

he puesto una pagina web estatica llamada `index.html` como aprendimos ayer esto muestra la `/` directamente este es nuestro html

```
<html>
    <body>
        <form action="/mostrar" method="post">
            <label for="nombre">Nombre:</label>
            <input type="text"  name="nombre">
            <button type="submit">Enviar</button>
        </form>
    </body>
```

y este es el `@Postmapping` donde procesamos y mostramos el el mensajes:

```
package ruvik.ud1ud2;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;


@SpringBootApplication
@RestController
public class Ud1Ud2Application {

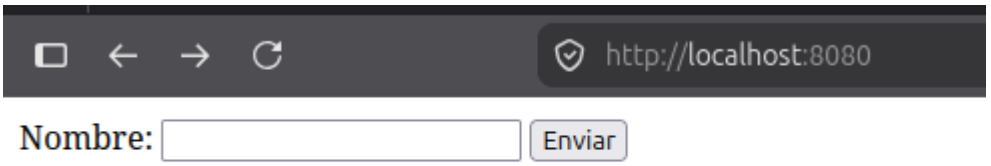
    public static void main(String[] args) {
        SpringApplication.run(Ud1Ud2Application.class, args);
    }

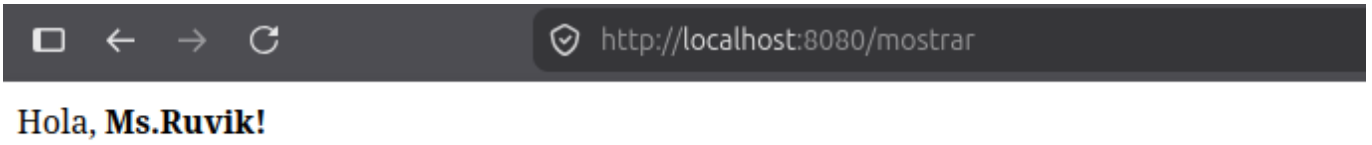
    @GetMapping("/hola")
    public String hello(@RequestParam(value = "nombre", defaultValue =
"Mundo") String name) {
        return String.format("Hola, <b> %s!</b>",name);
    }

    @PostMapping("/mostrar")
    public String mostrar(@RequestParam(value = "nombre", defaultValue =
"Mundo") String name) {
        return String.format("Hola, <b> %s!</b>",name);
    }

}
```

Y aqui las capturas de como se muestra un formulario en `/` y el resultado





Primero si quitamos el Rest de RestController y lo dejamos como un Controller a secas. Lo que pasas es que a parte de que la importanción cambia, aunque te siga sirviendo el html en la pagina principal, cuando quieres procesar algun tipo de dato falla. Por el hecho de que solo es un controller.

Pero no me acuerdo como era el año pasado, pero si he averiguado por que me sonaba entre `@RequestBody` y `@ResponseBody` pregunte en internet sus diferncias y el `@ResponseBody` es lo que devuelve el servidor, si pones la anotación arriba de lo que quieres mostrar termina funcionando.

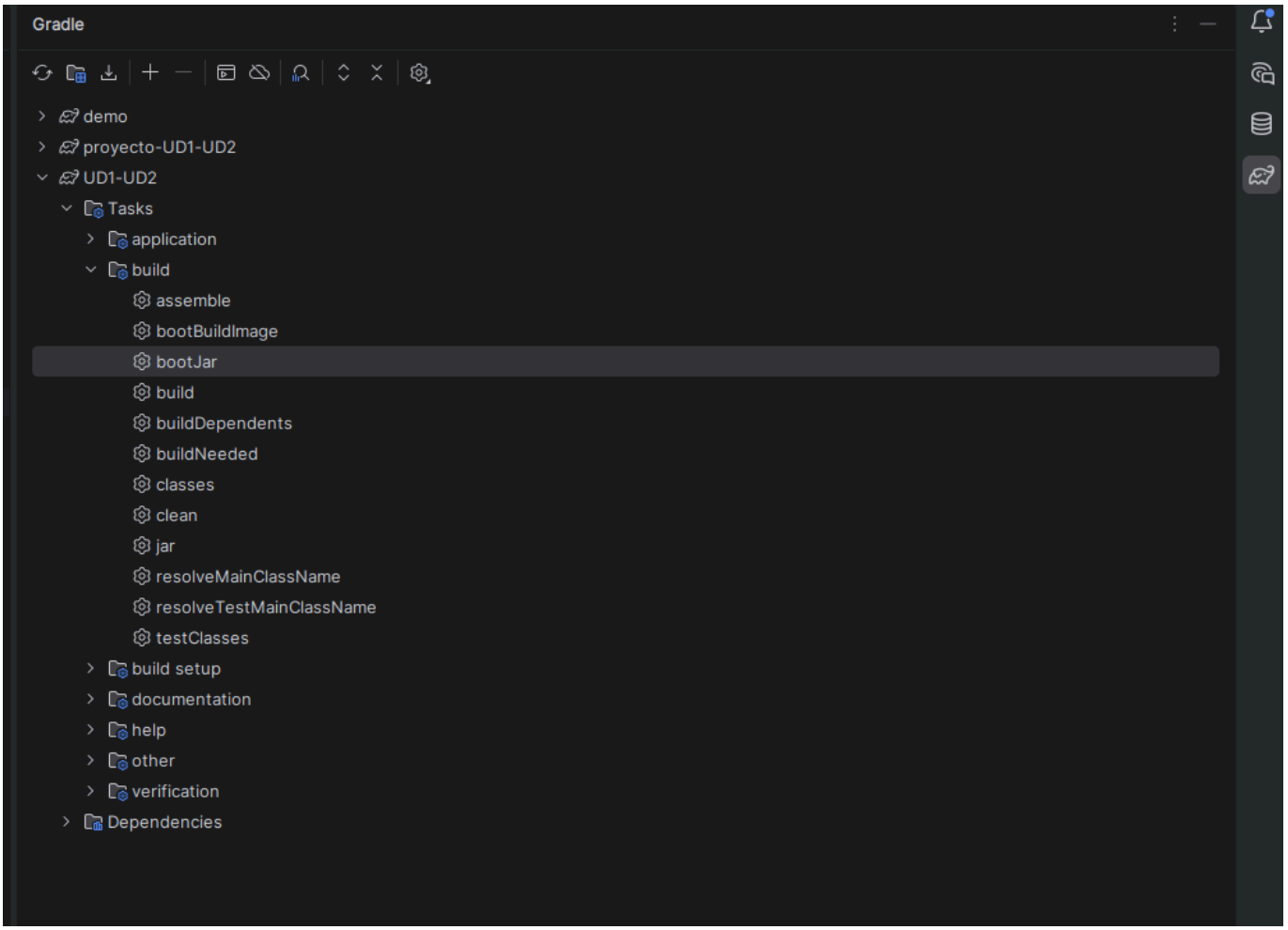
```
    @PostMapping("/mostrar")
    @ResponseBody
    public String mostrar(@RequestParam(value = "nombre", defaultValue =
"Mundo") String name) {
        return String.format("Hola, <b> %s!</b>",name);
    }
```

Sring Initializr

Lo que aporta es quitarle complijidad al crear una aplicación web que recibe peteciones, porque al utilizar el Spring Core, debemos tener muchas dependecias y ver si clases esta cableadas lo mejor posible, sino nada funciona.

Spring Initializr soluciona eso, dando una interfaz mas amable, que nos deja elegir todo y crear todo apps sin nosotros tener que preocuparnos que funciona o no, solo programar.

En intellJ para poder exportar el .jar es de los siguiente forma




```
vas al proyecto
            UD1-UD2
                Tasks
                    build
                    BootJar
```

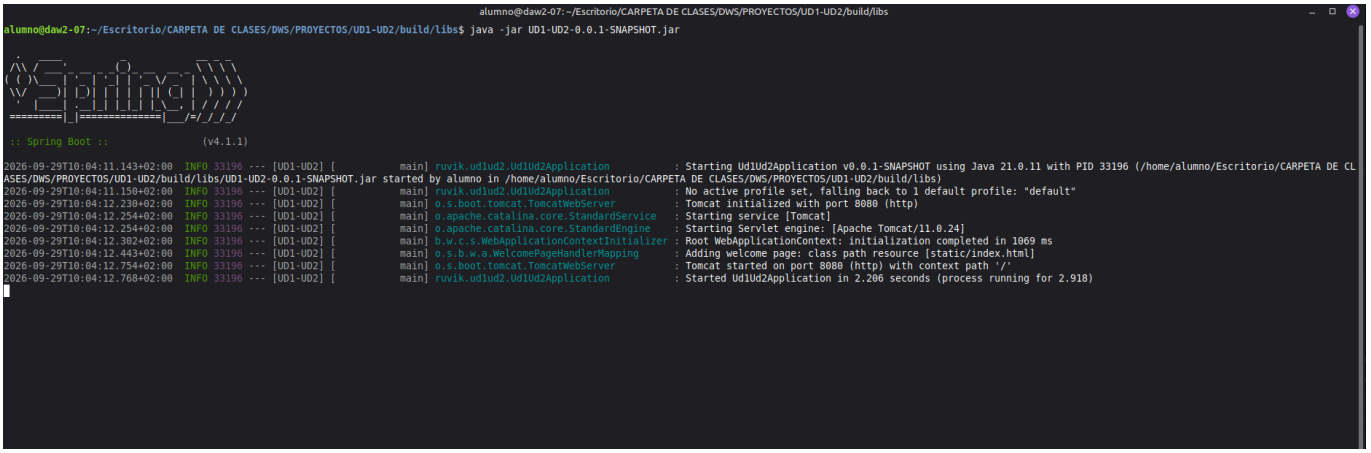
le damos doble click y ejecuta, en la carpeta e tu proyecto esta en build y lib

para ejecutarlo debemos ir ahí por terminal

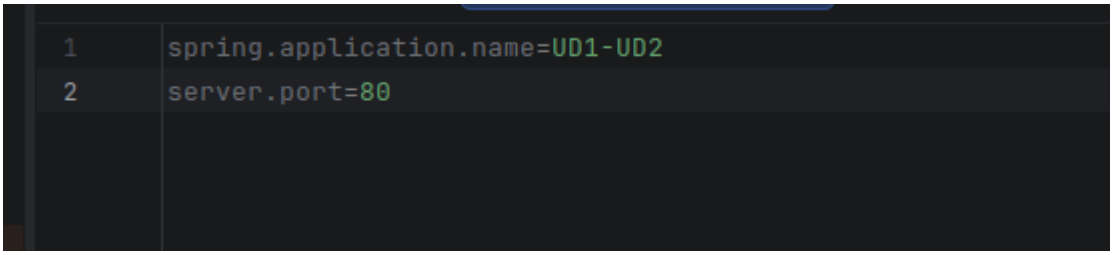
y ejecutar

```
java -jar [NOMBRE DE LA CARPETA]
```

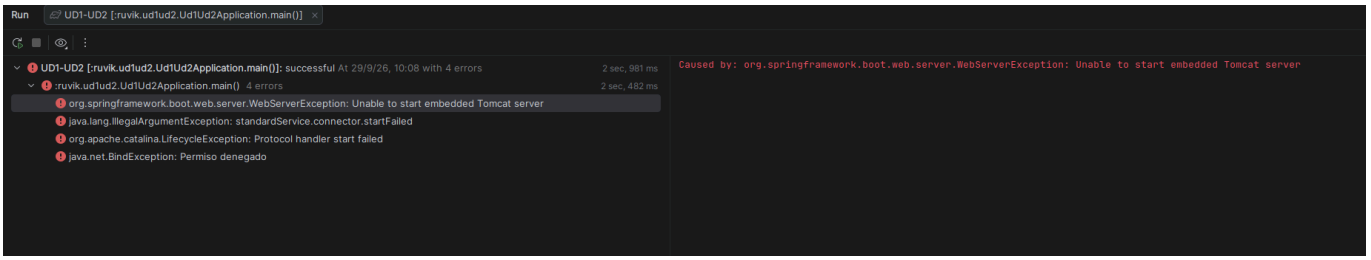
resultado



He buscado por internet y dice que el puerto se limita en `application.properties` he intento incluir el puerto:80



pero parece ser que todo salta por los aires.



Vale con el POJO introducido, devuelvo directamente el Objeto, porque habia pensado en devolver un tipo String, pero al devolver el Objeto, lo que hace Springboot es lo siguiente

```java
package ruvik.ud1ud2;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.web.bind.annotation.*;
import ruvik.ud1ud2.Entity.Contacto;


@SpringBootApplication
@RestController
public class Ud1Ud2Application {

    public static void main(String[] args) {
```

```
        SpringApplication.run(Ud1Ud2Application.class, args);
    }

    @GetMapping("/hola")
    public String hello(@RequestParam(value = "nombre", defaultValue =
"Mundo") String name) {
        return String.format("Hola, <b> %s!</b>",name);
    }

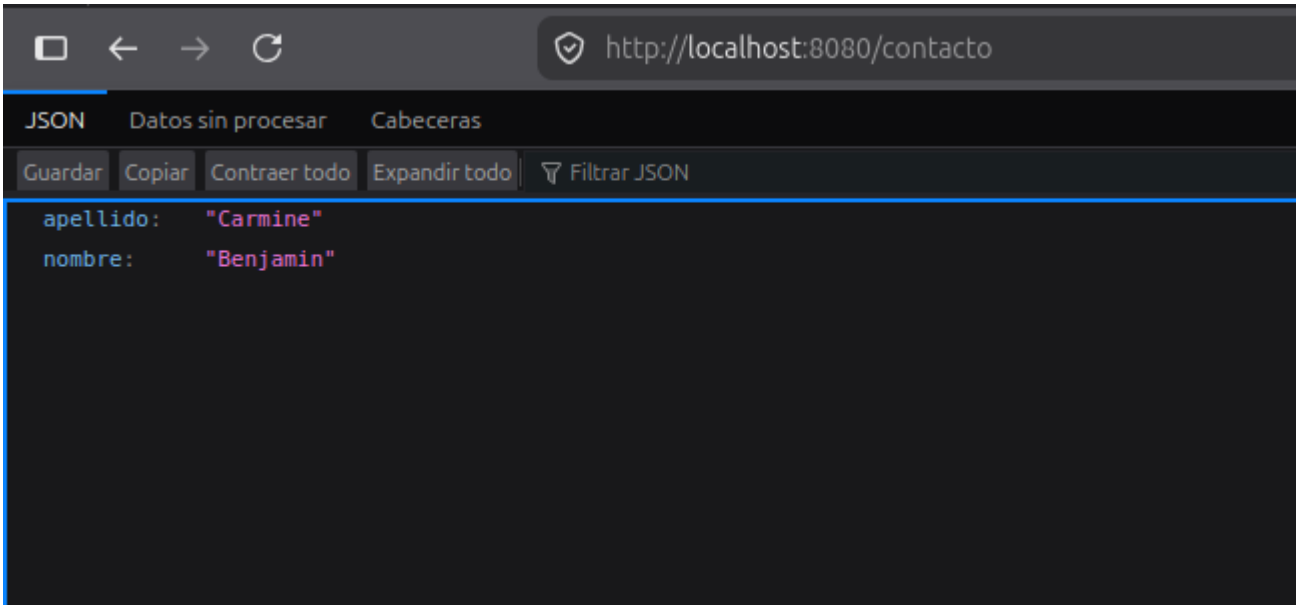
    @PostMapping("/mostrar")
    public String mostrar(@RequestParam(value = "nombre", defaultValue =
"Mundo") String name) {
        return String.format("Hola, <b> %s!</b>",name);
    }

    @GetMapping("/contacto")
    public Contacto contacto(@RequestParam(value = "nombre",defaultValue =
"Benjamin") String nombre,
                             @RequestParam(value = "apellido",defaultValue
= "Carmine") String apellido
    ){

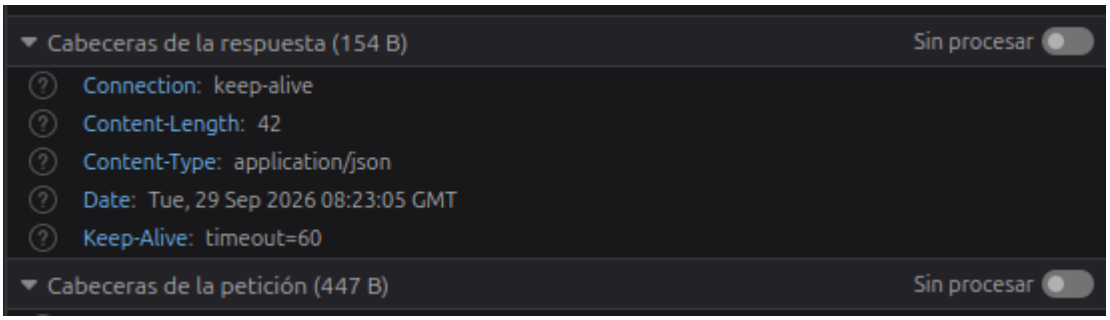
        Contacto contacto = new Contacto();
        contacto.setNombre(nombre);
        contacto.setApellido(apellido);
        return contacto;
    }

}
```

Y lo que interpreta Springboot es devolver un JSON



Y el Type al inspeccionar es el siguiente:

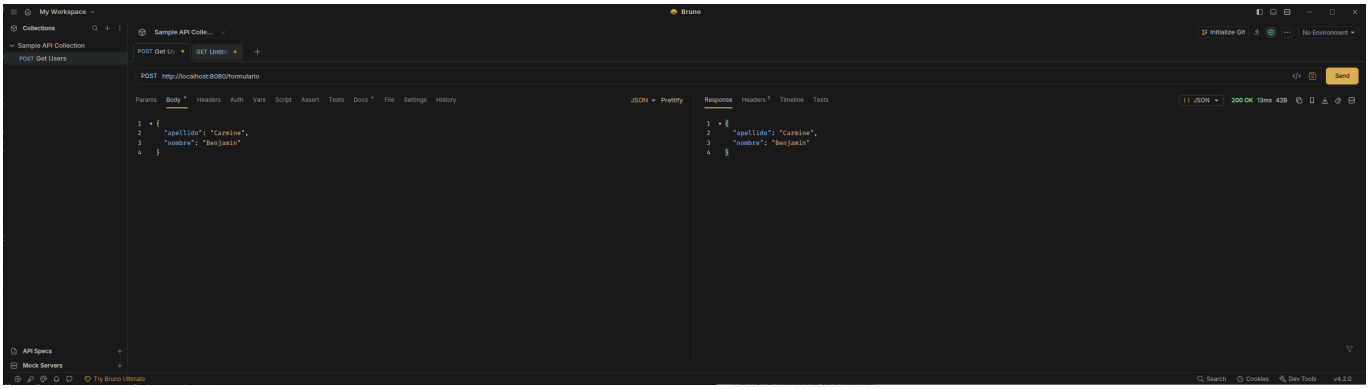


---

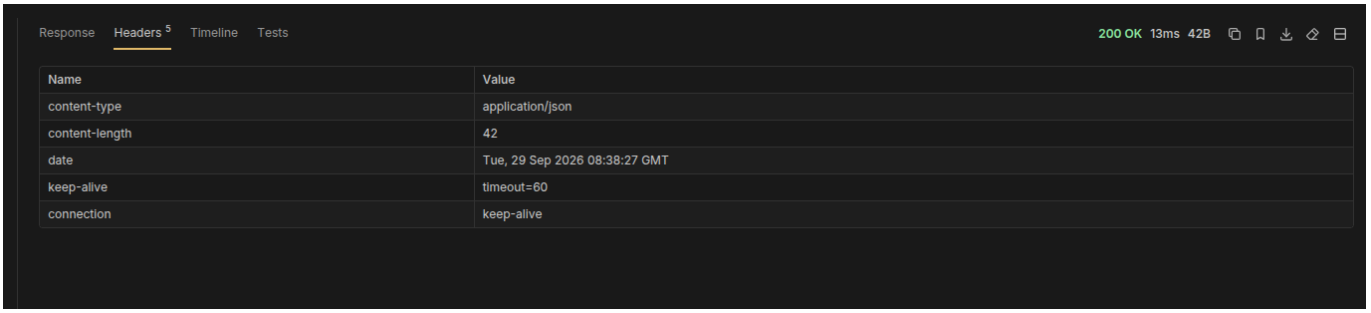
para utilizar bruno e ingresar datos por dentro del cuerpo, yo hice el `@PostMapping` lo siguiente:

```
    @PostMapping("/formulario")
    public Contacto formulario(@RequestBody(required = false) Contacto
contacto) {
        Contacto contacto1 = contacto;
        System.out.println("Si funciono, creo...");
        System.out.println(contacto1);
        return contacto1;
    }
```

Y en bruno hace lo siguiente:



Y la cabecera que devuelve es la siguiente:



Buscando información por internet, dice que se Gesntiona cuando utilizamos `@RestController` o `@ResponseBody` o cuando estemos mucho mas avanzados y utilicemos

`ResponseEntity` podemos de alguna manera configurar y gestionar el typeContent.

Conclusión

realmente el hecho de volver a reever esto, me hizo ir mas rapido, ya entendia el porque de algunas cosas, como funciona, es verdad que me senti oxidado mas con el tema de POJOS pero en general me he sentido mas suelto y capaz de hacerlo mejor, cuando lo intente por primera vez.

FINAL